

TECH CHALLENGE – FASE 3

Assistente Médico com LLM, LangChain e RAG

Relatório técnico final

Jefferson Antônio Pantoja Silva

12 de setembro de 2026

Código-fonte no GitHub

github.com/JeffersonPantoja/tech-challenge-fase3

Demonstração em vídeo

youtube.com/watch?v=n5sVF18qgVg

Abstract

Este relatório descreve o desenvolvimento de um assistente médico experimental para a Fase 3 do Tech Challenge. A solução integra preparação de dados biomédicos, geração de prontuários sintéticos, fine-tuning de uma LLM local e um fluxo de recuperação aumentada por geração (RAG), orquestrado por LangChain e LangGraph. O dataset é construído a partir de MedQuAD e PubMedQA, com normalização, curadoria, deduplicação e rastreabilidade de origem. A aplicação recupera informações de prontuários estruturados usando embeddings e FAISS, contextualiza a pergunta e gera a resposta com o modelo local ajustado. Também foram implementados tradução multilíngue, revisão automática de segurança, identificação das fontes consultadas e observabilidade opcional com Langfuse. O trabalho apresenta a arquitetura, as decisões técnicas, os limites de atuação e uma avaliação quantitativa do modelo fine-tuned em amostra separada de teste. Os resultados mostram boa similaridade semântica geral, mas baixa correspondência literal, reforçando a necessidade de avaliação clínica complementar.

Palavras-chave: inteligência artificial na saúde; fine-tuning; RAG; LangChain; LangGraph; segurança; prontuários sintéticos.

Contents

1	Introdução	1
2	Objetivos	1
2.1	Objetivo geral	1
2.2	Objetivos específicos	1
3	Fundamentação e requisitos	2
4	Materiais e métodos	2
4.1	Fontes de dados	2
4.2	Curadoria e formato do dataset	2
4.3	Geração de prontuários sintéticos	3
4.4	Fine-tuning	4
5	Arquitetura da solução	5
5.1	Pipeline completo	5
5.2	Consulta a prontuários	6
5.3	Fluxo LangChain e LangGraph	6

6	Segurança, validação e explainability	7
6.1	Observabilidade	7
7	Avaliação	8
7.1	Verificações realizadas	8
7.2	Protocolo de avaliação	8
7.3	Resultados e limitações	9
8	Reprodutibilidade e execução	10
9	Conclusão	10
	Anexos	12
	Anexo A – Execução do assistente médico	12
	Anexo B – Observabilidade com Langfuse	13
	Anexo C – Avaliação do modelo fine-tuned	13

1 Introdução

O desafio da Fase 3 propõe a criação de um assistente virtual médico treinado com conhecimento médico específico, capaz de apoiar dúvidas clínicas e consultar informações atualizadas do paciente. A proposta também exige fluxos automatizados e seguros, integração com LangChain, preparação e anonimização dos dados, logging, explainability e organização modular do código [1].

O projeto desenvolvido responde a esse desafio por meio de uma arquitetura em etapas. Primeiro, dados públicos de perguntas e respostas biomédicas são convertidos em um dataset textual compatível com treinamento supervisionado. Em seguida, os casos curados podem ser convertidos em prontuários fictícios estruturados. Por fim, o assistente combina uma LLM local ajustada com recuperação semântica dos prontuários, tradução de entrada e saída, revisão automática e rastreamento de execução.

O sistema não pretende substituir a decisão clínica. Sua função é fornecer apoio informacional contextualizado, preservando a necessidade de validação humana, especialmente diante de perguntas sobre diagnóstico, medicamentos ou tratamento.

2 Objetivos

2.1 Objetivo geral

Desenvolver e documentar um pipeline modular de assistente médico que integre preparação de dados, fine-tuning de LLM, consulta a prontuários sintéticos e mecanismos de segurança e observabilidade.

2.2 Objetivos específicos

- Ler e integrar as bases MedQuAD e PubMedQA em um dataset supervisionado.
- Aplicar normalização, validação, curadoria, deduplicação e preservação da origem dos registros.
- Gerar prontuários sintéticos em JSONL, com processamento em lotes, validação e retomada por checkpoint.
- Ajustar o modelo `meta-llama/Llama-3.2-1B-Instruct` usando QLoRA e SFTTrainer.
- Consultar prontuários por similaridade semântica, com opção de filtro por paciente.

- Orquestrar tradução, recuperação, construção de contexto, geração e revisão em um grafo LangGraph.
- Registrar fontes e metadados de execução sem expor conteúdo clínico integral por padrão.

3 Fundamentação e requisitos

A especificação da Fase 3 organiza a entrega em quatro preocupações técnicas: fine-tuning com dados médicos, assistente com LangChain e base estruturada, segurança e validação, e organização modular do projeto. Também exige um dataset anonimizado ou sintético, um relatório técnico, um diagrama do fluxo e uma avaliação do modelo [1].

Como as fontes adotadas são bases públicas destinadas a pesquisa, o projeto priorizou curadoria técnica e geração de dados sintéticos para o cenário de prontuários. Essa escolha evita inserir dados reais de pacientes na demonstração. Os identificadores dos prontuários sintéticos são normalizados sequencialmente e a origem de cada caso é preservada no campo `source`.

4 Materiais e métodos

4.1 Fontes de dados

Foram utilizadas duas bases públicas [7, 8], alinhadas às referências de datasets registradas na wiki do projeto:

- **MedQuAD**: arquivos XML com foco temático e pares de pergunta e resposta provenientes de fontes do NIH.
- **PubMedQA**: arquivos JSON com pergunta biomédica, contextos, rótulos e, quando disponível, resposta longa.

O leitor `MedQaSourcesReader` percorre os XML do MedQuAD e os JSON do PubMedQA. Para MedQuAD, o campo `Focus` é usado como contexto e cada par recebe uma origem no formato `MedQuAD:arquivo:pid`. Para PubMedQA, a resposta longa é preferida aos rótulos, enquanto os contextos são concatenados. A origem assume o formato `PubMedQA:arquivo:id`.

4.2 Curadoria e formato do dataset

O serviço `QARecordCurationService` executa as seguintes operações:

1. Remove espaços redundantes e normaliza os campos textuais;
2. Descarta registros sem origem, pergunta ou resposta;
3. Descarta perguntas ou respostas com menos de oito caracteres;
4. Elimina duplicidades pela assinatura composta de pergunta, resposta e contexto;
5. Registra estatísticas de itens mantidos e descartados por motivo.

O resultado é escrito progressivamente em JSONL. O campo `text` segue o template que será reutilizado na inferência:

ANSWER THE QUESTION.

[|Context|] ... [|eContext|]

[|Question|] ... [|eQuestion|]

[|Answer|] ... [|eAnswer|]

A leitura incremental e a escrita progressiva reduzem o uso de memória. Essa separação também mantém os leitores de fonte e a curadoria independentes de LangChain, concentrando a abstração de orquestração na etapa em que ela é necessária.

4.3 Geração de prontuários sintéticos

Cada registro curado pode ser enviado em lote a um gerador baseado na API da OpenAI ou em uma LLM local. O retorno esperado é um vetor JSON com exatamente um item por origem e as chaves `source`, `patient_id`, `chief_complaint`, `history`, `medications`, `vitals`, `assessment` e `plan`.

O lote é validado pela correspondência dos valores de `source`. Se houver item ausente, excedente ou duplicado, o lote é reprocessado até três vezes. Após as tentativas, um lote inválido é descartado e o fluxo continua. Os registros válidos são gravados incrementalmente e um checkpoint por origem permite retomar uma execução sem repetir lotes concluídos.

Após a geração, o controlador de normalização substitui os identificadores por valores sequenciais. O procedimento preserva os demais campos e facilita a seleção explícita de um paciente durante a consulta.

4.4 Fine-tuning

O notebook de treinamento utiliza o modelo base `meta-llama/Llama-3.2-1B-Instruct`, carregado em 4 bits com Unsloth. A adaptação é feita com QLoRA, técnica de ajuste eficiente em memória, seguindo a trilha de preparação e fine-tuning da documentação de referência [2, 3, 9]. Os parâmetros principais são apresentados na Tabela 1.

Table 1: Parâmetros registrados no notebook de fine-tuning.

Parâmetro	Valor
Rank LoRA (r)	16
LoRA alpha	32
LoRA dropout	0,05
Batch por dispositivo	2
Acumulação de gradientes	8
Taxa de aprendizagem	2×10^{-4}
Épocas	1
Avaliação	ao final de cada época
Salvamento	a cada 100 passos
Precisão	BF16, ou FP16 quando necessário

O processo de ajuste fino combina QLoRA e o `SFTTrainer` para adaptar o modelo de linguagem com menor consumo computacional. Em vez de atualizar todos os parâmetros do modelo original, a maior parte dos pesos é mantida congelada e são acrescentados adaptadores LoRA, formados por matrizes de baixa dimensão treináveis. Dessa forma, somente uma fração dos parâmetros é ajustada para incorporar o comportamento desejado, reduzindo o custo de memória e armazenamento em comparação com o fine-tuning completo.

A quantização é realizada pelo carregamento do modelo base em 4 bits. Essa representação reduz a memória necessária para manter os pesos do modelo durante o treinamento e torna a execução viável em uma GPU de ambiente Colab. Os adaptadores LoRA são aplicados às projeções internas do modelo, enquanto os pesos quantizados do modelo base permanecem preservados. O resultado é um modelo especializado por meio de uma quantidade reduzida de parâmetros treináveis, sem a necessidade de modificar integralmente a arquitetura original.

O treinamento supervisionado é conduzido pelo `trl.SFTTrainer`, que organiza a apresentação dos exemplos, calcula a função de perda e atualiza os parâmetros treináveis dos adaptadores. Cada registro é representado como um texto contendo instrução, pergunta, contexto quando disponível e resposta esperada. O modelo aprende, assim, a produzir uma continuação adequada para a pergunta médica a partir desse padrão textual.

A combinação entre quantização, adaptadores LoRA e treinamento supervisionado permite especializar a LLM para o domínio biomédico mantendo requisitos computacionais compatíveis com o ambiente utilizado.

O dataset é dividido em treino e teste com `train_test_split`. O treinamento é executado pelo `trl.SFTTrainer`, com possibilidade de retomar o último checkpoint. Ao final, o adapter é mesclado ao modelo e o diretório resultante deve conter os arquivos necessários para inferência, incluindo configuração, pesos e tokenizer.

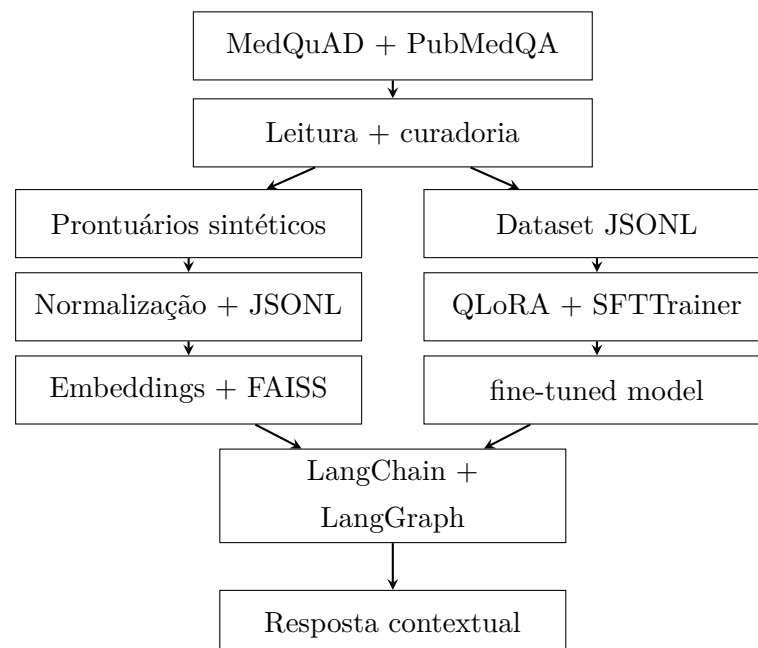
5 Arquitetura da solução

O código está organizado em camadas inspiradas em Clean Architecture:

- **domain:** modelos imutáveis, como `QARecord` e `PatientRecordDocument`;
- **application:** casos de uso e portas, como construção do dataset e consulta RAG;
- **infrastructure:** leitores, escritores, modelos locais, embeddings, integrações OpenAI e Langfuse;
- **presentation:** controladores de linha de comando para os diferentes fluxos.

Essa divisão reduz o acoplamento entre regras de negócio e bibliotecas de infraestrutura. Por exemplo, o caso de uso RAG depende de uma porta de recuperação, e não diretamente da implementação FAISS.

5.1 Pipeline completo



5.2 Consulta a prontuários

O leitor `JsonPatientRecordDocumentReader` valida que cada registro possui uma origem e que não existem fontes duplicadas. O conteúdo indexado inclui identificador, queixa principal, histórico, medicamentos, sinais vitais e avaliação. O campo `plan` permanece armazenado no JSONL, mas é deliberadamente excluído do índice e do contexto enviado ao modelo, reduzindo o risco de a LLM reproduzir automaticamente uma conduta de tratamento.

O retriever `FaissPatientRecordRetriever` utiliza `sentence-transformers/all-MiniLM-L6-v2` para embeddings e FAISS para busca semântica, conforme a abordagem de recuperação de contexto apresentada na aula de RAG [4, 10]. Sem filtro, são recuperados até quatro documentos relevantes. Com `patient_id`, a busca é restrita aos documentos daquele paciente e depois ranqueada semanticamente.

5.3 Fluxo LangChain e LangGraph

O fluxo implementado pelo caso de uso `AskMedicalAssistantWithRagUseCase` é apresentado na Figura 1. A composição de prompts e etapas encadeadas segue os conceitos de LangChain, enquanto os nós e transições de estado seguem o padrão de `StateGraph` estudado nos materiais de LangGraph [5, 6].

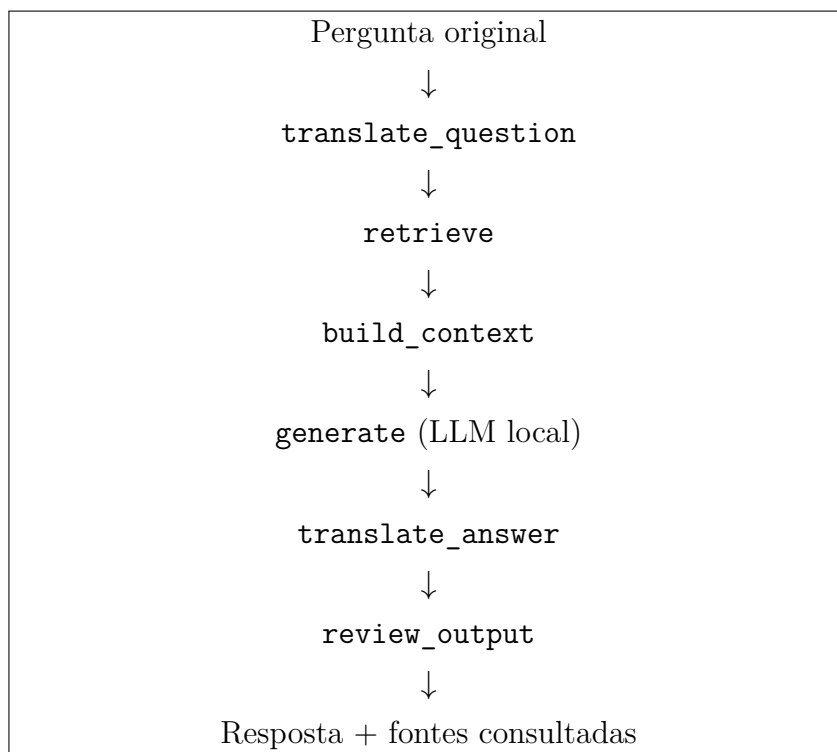


Figure 1: Grafo de contextualização e revisão da resposta médica.

O template de prompt usado na geração é compatível com o template do fine-tuning. A pergunta em português pode ser traduzida para inglês antes da recuperação, pois o treinamento, os prontuários sintéticos e o modelo de embeddings foram definidos inicialmente nesse idioma. Após a geração local, a resposta é traduzida para o idioma original.

6 Segurança, validação e explainability

A segurança foi tratada como uma camada adicional, e não como uma propriedade presumida da LLM. O revisor `OpenAIMedicalResponseReviewer` analisa a pergunta, o contexto e a resposta. Quando há pedido ou instrução sobre medicamento ou tratamento, a saída é substituída por uma orientação de revisão por profissional especializado. Nos demais casos, a resposta é mantida e recebe um parágrafo de revisão automática.

Se a chamada de revisão falhar, existe um fallback determinístico que mantém a resposta e acrescenta a advertência de que diagnóstico e tratamento devem ser avaliados por profissional de saúde. Essa estratégia evita que a indisponibilidade de um componente externo interrompa completamente a aplicação, sem transformar a resposta em prescrição.

As fontes são retornadas junto com a resposta no formato que associa o caminho de origem ao identificador do paciente. Assim, o usuário pode rastrear quais registros foram recuperados. O sistema não apresenta a LLM como autoridade clínica e não usa o campo `plan` no contexto de geração.

Uma execução do assistente médico é apresentada no **Anexo A**. O terminal apresenta perguntas contextualizadas, manutenção do paciente na sessão, revisão automática e as fontes recuperadas para cada resposta.

6.1 Observabilidade

O Langfuse é opcional e self-hosted. Quando as credenciais estão configuradas, o trace registra o identificador da requisição, o paciente selecionado, as etapas do grafo, a geração local e as chamadas de tradução e revisão. Por padrão, `LANGFUSE_CAPTURE_CONTENT=false`, de modo que o sistema registra tamanhos e contagens de tokens sem capturar integralmente o conteúdo clínico. A captura integral só deve ser ativada em ambiente controlado.

Também são registrados tokens de entrada, saída e total. No modelo local, a contagem é feita pelo tokenizer; nas chamadas OpenAI, utiliza-se o campo de uso retornado pela API. Se o Langfuse não estiver disponível, o assistente continua executando sem tracing.

A evidência visual do trace registrado no Langfuse está disponível no **Anexo B**. A interface evidencia a execução do grafo, a geração pelo modelo local, a latência e a

contagem de tokens do prompt e da conclusão.

7 Avaliação

7.1 Verificações realizadas

O repositório fornece comandos de verificação estática:

```
python3 -m compileall src
```

Também foram verificadas, por inspeção da implementação, as propriedades abaixo:

- o dataset mantém a origem de cada item;
- a curadoria contabiliza registros mantidos, vazios, curtos e duplicados;
- os lotes de prontuários exigem exatamente uma ocorrência por origem;
- o leitor de prontuários rejeita fontes ausentes ou duplicadas;
- o retriever permite restrição por paciente;
- o contexto não inclui o campo **plan**;
- o grafo executa as etapas em ordem e retorna as fontes recuperadas;
- a revisão impede que a resposta seja apresentada como prescrição direta;
- o tracing não captura conteúdo clínico por padrão.

7.2 Protocolo de avaliação

O notebook executa a avaliação do modelo fine-tuned, sem inferência ou comparação com o modelo base, seguindo estas etapas:

1. separar um conjunto de perguntas de teste que não seja usado no treinamento;
2. medir a perda de avaliação do **SFTTrainer** no conjunto de teste;
3. gerar respostas em batches para uma amostra aleatória de 500 exemplos do conjunto de teste;
4. calcular Exact Match, Token F1, ROUGE-L e BERTScore F1 com bibliotecas especializadas;

5. criar casos de consulta com e sem filtro de paciente;
6. verificar se a fonte recuperada corresponde ao paciente e à pergunta;
7. testar perguntas em inglês e português, verificando tradução e retorno no idioma original;
8. exercitar perguntas sobre medicamentos e tratamento para confirmar o bloqueio de segurança;
9. testar indisponibilidade da revisão e do Langfuse;
10. avaliar manualmente factualidade, relevância, ausência de prescrição e rastreabilidade.

7.3 Resultados e limitações

Foi realizada uma avaliação sobre 500 exemplos selecionados aleatoriamente do conjunto de teste. Os resultados foram:

Table 2: Resultados da avaliação do modelo fine-tuned.

Métrica	Resultado
Exact Match	0,0000
Token F1	0,3280
ROUGE-L	0,2717
BERTScore F1	0,8354

O Exact Match igual a zero e os valores moderados de Token F1 e ROUGE-L indicam baixa correspondência literal entre as respostas geradas e as referências. O BERTScore F1 de 0,8354 indica boa similaridade semântica geral, mas não garante correção factual. A inspeção qualitativa mostrou exemplos semanticamente próximos e também pelo menos um desvio temático relevante, o que impede interpretar o resultado como validação clínica. Não foram medidos neste experimento métricas de recuperação, perda final registrada ou avaliação por especialistas.

Durante o carregamento do modelo usado pelo BERTScore, pesos da camada de language modeling foram reportados como `UNEXPECTED`. O aviso é esperado quando se carrega uma arquitetura de tarefa diferente para obter o modelo base de embeddings e não impediu o cálculo das métricas.

As principais limitações são: dependência de GPU para treinamento e inferência prática; dependência da API OpenAI para tradução, geração sintética e revisão; qualidade condicionada aos dados públicos; embeddings inicialmente orientados ao inglês; e ausência

de validação clínica formal. Essas limitações devem ser resolvidas antes de qualquer uso assistencial real.

8 Reprodutibilidade e execução

O ambiente recomendado utiliza Python 3.12 ou superior e um ambiente virtual. A preparação básica é:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Para gerar o dataset:

```
python3 -m src.main --resources-dir resources \\  
--output resources/finetuning_qa.jsonl
```

O notebook `src/notebooks/fine-tuning-colab.ipynb` executa o ajuste no Google Colab. Após salvar o modelo mesclado em `resources/medqa-finetuned-model/` e disponibilizar os prontuários em `resources/patient_records.jsonl`, o assistente RAG pode ser iniciado com:

```
python3 -m src.main_rag
```

Os notebooks de fine-tuning, assistente médico e geração de prontuários documentam as etapas que dependem de recursos externos. O README do projeto concentra as variáveis de ambiente, caminhos, comandos e opções de execução.

9 Conclusão

O projeto implementou uma solução modular para o desafio de construir um assistente médico com conhecimento biomédico ajustado e contexto de prontuários. O pipeline cobre a preparação de dados MedQuAD e PubMedQA, a curadoria com rastreabilidade, a geração de prontuários sintéticos, o fine-tuning com QLoRA, a recuperação FAISS e a orquestração LangChain/LangGraph.

Também foram incorporadas salvaguardas relevantes para o domínio: ausência do campo de plano terapêutico no contexto RAG, revisão automática de respostas, bloqueio de instruções diretas de tratamento, identificação das fontes e observabilidade configurável sem captura de conteúdo por padrão. Essas medidas reduzem riscos, mas não eliminam a necessidade de supervisão humana.

Como próximos passos, a prioridade é registrar a perda final, medir as métricas de recuperação, ampliar a avaliação para outros tamanhos de amostra, conduzir avaliação qualitativa especializada e testar o comportamento em cenários de erro. A solução deve permanecer como apoio à decisão, nunca como substituta da responsabilidade de profissionais de saúde.

Anexo A – Execução do assistente médico

```
tech-challenge-fase-3: python3 — Konsole
```

Nova aba | Dividir a exibição | Copiar | Colar | Localizar

```
tech-challenge-fase-3: opencode • tech-challenge-fase-3: python3 • tech-challenge-fase-3: bash •
```

Loading weights: 100%|██████████| 146/146 [00:03<00:00, 43.31it/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 4808.86it/s]
Prontuários indexados: 35644
Digite uma pergunta, 'sair' para encerrar ou '/clear' para limpar o paciente atual.

Pergunta: Quais são os paciente com sarampo?
Patient ID (opcional):

Resposta: O paciente apresentou febre, tosse e erupção cutânea. O histórico de sarampo do paciente e sinais de declínio cognitivo sugerem um possível diagnóstico de SSPE.

Revisão automática: O paciente foi diagnosticado com sarampo anteriormente e apresenta indicadores de declínio cognitivo segundo o registro. O diagnóstico e tratamento devem ser avaliados por um profissional de saúde especializado.
Fontes: PubMedQA:ori_pqau.json:22888711 (ID do paciente: 28929), MedQuAD:9_CDC_QA/0000199.xml:14 (ID do paciente: 654), MedQuAD:4_MPlus_Health_Topics_QA/0000585.xml:1 (ID do paciente: 14849), MedQuAD:6_NINDS_QA/0000095.xml:1 (ID do paciente: 14467)

Pergunta: Quais são os sintomas do paciente?
Patient ID (opcional): 14467

Resposta: A paciente é uma criança com histórico de sarampo. A paciente tem histórico d e declínio neurológico. A pressão arterial da paciente está elevada, a frequência cardíaca está elevada, a frequência respiratória está elevada e a temperatura está elevada. A paciente tem 10 anos.

Revisão automática: A paciente foi avaliada e a condição é uma possibilidade de acordo com o registro. A avaliação de diagnóstico e tratamento deve ser feita por um profissional de saúde especializado.
Fontes: MedQuAD:6_NINDS_QA/0000095.xml:1 (ID do paciente: 14467)

Pergunta: Qual é o tratamento ou medicação adequado?
Patient ID [14467] (Enter para manter):

Resposta: Revisão automática: A solicitação envolve medicação ou tratamento. Essas orientações devem ser avaliadas por um profissional de saúde especializado.
Fontes: MedQuAD:6_NINDS_QA/0000095.xml:1 (ID do paciente: 14467)

Pergunta: Quais são os principais sintomas do sarampo?
Patient ID [14467] (Enter para manter):

Resposta: A principal sintoma do sarampo é uma erupção cutânea que começa no rosto e se espalha para outras partes do corpo. Além disso, os pacientes podem apresentar febre, dor de cabeça, dores musculares, dor nas articulações e fadiga. Esses sintomas geralmente aparecem dentro de 3 dias após a infecção e se resolvem em 7-14 dias.

Revisão automática: O paciente já foi diagnosticado com sarampo e apresenta sinais de declínio cognitivo, segundo o registro. O diagnóstico e tratamento devem ser avaliados por um profissional de saúde especializado.
Fontes: MedQuAD:6_NINDS_QA/0000095.xml:1 (ID do paciente: 14467)

Pergunta: /clear
Paciente atual removido.

Pergunta: tem pacientes com dengue?
Patient ID (opcional):

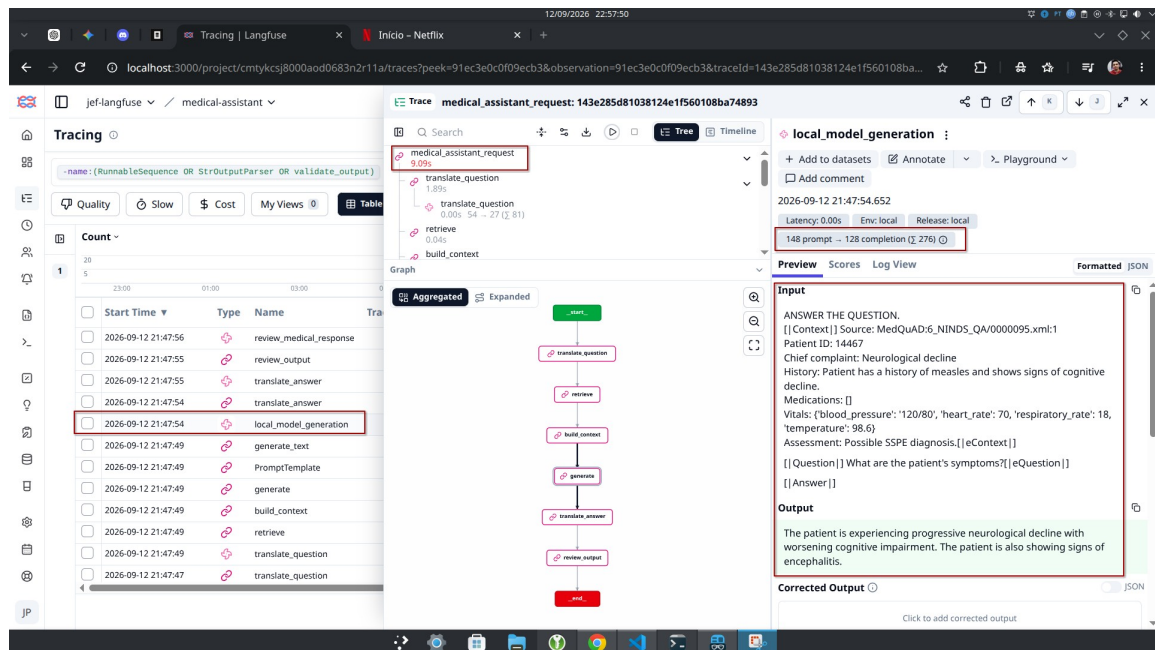
Resposta: A dengue é um problema significativo de saúde pública em muitos países tropicais. A Organização Mundial da Saúde (OMS) recomenda que todos os viajantes para áreas endêmicas devam ser vacinados contra a febre dengue. Além disso, os viajantes devem ser aconselhados a evitar picadas de mosquito e evitar áreas onde os mosquitos estão presentes. Os viajantes também devem estar cientes dos sinais e sintomas da febre dengue e procurar atendimento médico se desenvolverem algum desses sintomas.

Revisão automática: O paciente foi avaliado e a condição é uma possibilidade de acordo com o registro. O diagnóstico e o tratamento devem ser avaliados por um profissional de saúde especializado.
Fontes: MedQuAD:4_MPlus_Health_Topics_QA/0000261.xml:1 (ID do paciente: 15480), PubMedQA:ori_pqau.json:23675548 (ID do paciente: 18071), PubMedQA:ori_pqau.json:15756042 (ID do paciente: 32894), PubMedQA:ori_pqau.json:22768874 (ID do paciente: 33609)

Pergunta:

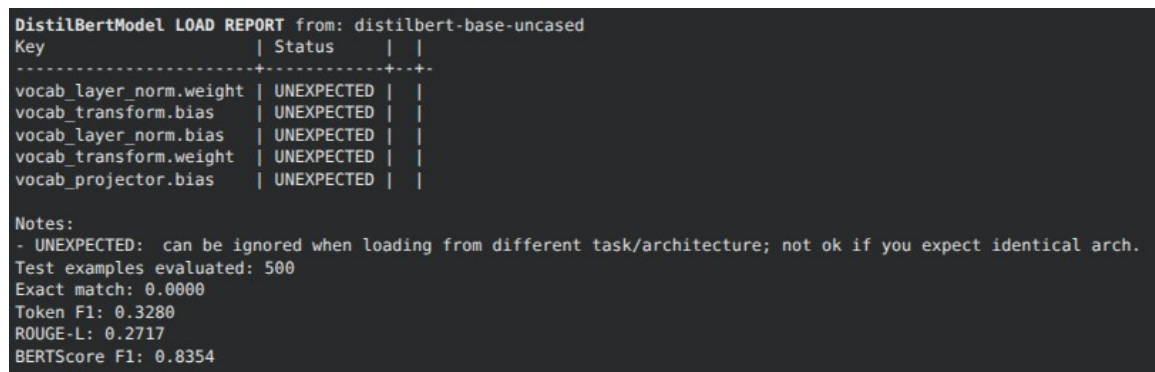
Execução do assistente médico no terminal, com respostas revisadas e fontes consultadas.

Anexo B – Observabilidade com Langfuse



Trace do Langfuse com o fluxo LangGraph e a geração do modelo local.

Anexo C – Avaliação do modelo fine-tuned



Resultado da avaliação do modelo fine-tuned no conjunto de teste.

References

- [1] POSTECH. *8IADT – Fase 3 – Tech Challenge*. Documento de especificação fornecido para a atividade. São Paulo: FIAP, 2026.
- [2] POSTECH. *Preparando dados de treinamento para fine-tuning*. Aula 01 da trilha Fine-tuning e RAG para documentos. Material didático local disponível na wiki de referência do projeto. São Paulo: FIAP, 2026.
- [3] POSTECH. *Fine-tuning de LLM para documentos*. Aula 02 da trilha Fine-tuning e RAG para documentos. Material didático local disponível na wiki de referência do projeto. São Paulo: FIAP, 2026.
- [4] POSTECH. *RAG para documentos*. Aula 03 da trilha Fine-tuning e RAG para documentos. Material didático local disponível na wiki de referência do projeto. São Paulo: FIAP, 2026.
- [5] POSTECH. *Materiais de aula LangChain*. Aulas de introdução, document loaders, prompts, chains, integração com LLM e agents. Wiki de referência do projeto. São Paulo: FIAP, 2026.
- [6] POSTECH. *Materiais de aula LangGraph*. StateGraph, nós, estado compartilhado e orquestração de fluxos. Wiki de referência do projeto. São Paulo: FIAP, 2026.
- [7] ABACHA, Asma Ben; DEMNER-FUSHMAN, Dina. *MedQuAD: Medical Question Answering Dataset*. Repositório GitHub. Disponível em: <https://github.com/abachaa/MedQuAD>. Acesso em: 12 set. 2026.
- [8] JIN, Qiao et al. PubMedQA: a research question answering dataset. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Hong Kong: Association for Computational Linguistics, 2019. Disponível em: <https://pubmedqa.github.io/>. Acesso em: 12 set. 2026.
- [9] DETTMERS, Tim et al. QLoRA: efficient finetuning of quantized LLMs. arXiv, 2023. Disponível em: <https://arxiv.org/abs/2305.14314>. Acesso em: 12 set. 2026.
- [10] JOHNSON, Jeff; DOUZE, Matthijs; JÉGOU, Hervé. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019. Disponível em: <https://arxiv.org/abs/1702.08734>. Acesso em: 12 set. 2026.